

Técnicas e Algoritmos em Ciência de Dados

Tarefa 4

Este trabalho deve ser enviado até o dia 14 de junho de 2026, às 23:59h.
Envios atrasados serão penalizados em 10% por hora de atraso.

Resultados de aprendizagem avaliados

Esta tarefa oferece aos alunos a oportunidade de aplicar modelos de classificação baseados em Florestas Aleatórias e de estudar, de forma prática, estratégias para tratar valores ausentes em árvores de decisão. A tarefa também revisita o aprendizado não supervisionado por meio da implementação do algoritmo K-means, com visualização do movimento dos centroides e análise da soma das distâncias quadradas.

Instruções

Identificador

Por favor, use o **mesmo número aleatório de 6 dígitos que você usou para as TAREFAS 1,2,3** e escreva-o na primeira célula do notebook. Certifique-se de manter uma cópia desse número, pois ele será usado para fornecer o feedback.

Submissão

Envie seus arquivos através do ECLASS. Os arquivos que você envia não podem ser substituídos por mais ninguém e não podem ser lidos por nenhum outro aluno. No entanto, você pode substituir seu envio quantas vezes quiser, reenviando, embora apenas a última versão enviada seja mantida.

Se você tiver problemas, no último minuto, envie sua tarefa por e-mail como um anexo em alberto.paccanaro@fgv.br com o assunto "URGENTE – SUBMISSÃO TAREFA 4". No corpo da mensagem, explique o motivo de não enviar através do ECLASS.

IMPORTANTE

- Seu envio consistirá em um único notebook do Python implementando suas soluções.
- **O nome do arquivo será o número aleatório que o identifica (por exemplo, 568423.ipynb)**
- Esta tarefa consiste em 2 partes. Certifique-se de que o código de ambas as partes é colocado nas células de código relevantes no notebook.
- **NÃO ENVIE NENHUM CONJUNTO DE DADOS**, apenas o código.
- Qualquer função auxiliar que você usará deve ser incluída no notebook – não envie scripts adicionais.
- *Por favor, tente não usar ferramentas de IA como ChatGPT, Copilot, Gemini, etc. Este trabalho foi elaborado para que você se familiarize com conceitos importantes que só podem ser aprendidos por meio da experiência prática.*

Todo o trabalho que você enviar deve ser exclusivamente seu próprio trabalho. As submissões de trabalhos do curso serão verificadas para isso.

Caso seja suspeitada alguma atividade irregular nas tarefas (por exemplo, plágio, uso de IA etc.), os laboratórios serão também avaliados e, para cada laboratório considerado inadequado, o aluno perderá o ponto correspondente.

Critérios de marcação

Este trabalho de curso é avaliado e obrigatório e vale 10% da sua nota final total para este curso. Para obter nota máxima para cada pergunta, você deve respondê-la corretamente, mas também completamente. Serão dadas notas para escrever código de estrutura de poço.

IMPORTANTE: Em geral, você pode usar *numpy* ou outras bibliotecas básicas para operações de matriz, mas você não pode usar qualquer função de biblioteca que implementaria alguns dos algoritmos que você é obrigado a implementar. Se você está em dúvida sobre uma função específica, envie-nos um e-mail.

Além disso, para o ajuste de parâmetros, você não tem permissão para usar qualquer função de pesquisa de “grid”, como GridSearchCV do sklearn ou funções equivalentes.

Tarefa IV

Parte 1 – Random Forest com valores ausentes (valor: 60%)

Neste exercício, você trabalhará com classificação supervisionada usando Florestas Aleatórias. Você pode reutilizar e adaptar as implementações de árvore de decisão e Random Forest feitas em laboratórios anteriores. O foco principal desta tarefa não é repetir a implementação básica desses modelos, mas modificar as árvores da floresta para lidar com valores ausentes durante a escolha das divisões, durante o treinamento e durante a predição.

Você usará o arquivo `horse_colic_missing.csv`, baseado no conjunto de dados Horse Colic. O “*target*” é `surgical_lesion`, em que 1 indica que o caso tinha lesão cirúrgica e 0 indica que não tinha. Use apenas atributos clínicos pré-desfecho. Não use `surgery`, `hospital_number`, `outcome`, códigos de lesão ou `cp_data`, pois esses campos são identificadores, variáveis de tratamento/desfecho ou informações que podem introduzir vazamento.

O conjunto contém colunas com alta proporção de valores ausentes. Por exemplo, `nasogastric_reflux_ph` tem mais de 80% de valores ausentes e `abdomcentesis_total_protein` tem mais de 60%.

Estratégias para valores ausentes

Você deve comparar Florestas Aleatórias que diferem principalmente na forma como suas árvores tratam valores ausentes. Abaixo estão as duas estratégias que devem ser consideradas.

1. *Estratégia simples de pré-processamento.* Implemente uma referência simples, como imputação por mediana/moda aprendida apenas no treino. Essa estratégia serve como baseline.
2. *Roteamento fracionário por frequência de ramo.* Ao avaliar uma divisão em um nó, considere apenas os `n_observed` exemplos que possuem um valor observado para esse atributo. Use esses exemplos para calcular os pesos a serem atribuídos aos exemplos com valor ausente que serão então enviados para os dois filhos. Os pesos são dados por:

```
p_left = n_left_observed / n_observed
p_right = n_right_observed / n_observed
```

Exemplo

Considere um nó contendo 100 exemplos. Desejamos dividir esses exemplos entre os ramos esquerdo e direito da árvore utilizando um determinado atributo. No entanto, esse atributo está disponível para apenas 60 dos 100 exemplos; os 40 exemplos restantes possuem valor ausente para esse atributo.

Ao aplicar o teste de divisão aos 60 exemplos que possuem valor observado, verificamos que:

- 40 exemplos seguem para o ramo esquerdo;
- 20 exemplos seguem para o ramo direito.

Com base nesses 60 exemplos, estimamos a probabilidade de um exemplo seguir cada ramo:

- ramo esquerdo: $(40/60 = 2/3)$;
- ramo direito: $(20/60 = 1/3)$.

Os 40 exemplos com valor ausente não são descartados. Em vez disso, cada um deles é enviado para ambos os ramos da árvore:

- com peso $(2/3)$ no ramo esquerdo;
- com peso $(1/3)$ no ramo direito.

Dessa forma, cada exemplo com valor ausente contribui parcialmente para ambos os ramos, de acordo com a distribuição observada nos exemplos que possuem o atributo.

Esses pesos são então utilizados nos cálculos realizados nos nós descendentes da árvore.

IMPORTANTE: A entropia, o ganho de informação e a frequência das classes nas folhas devem ser calculados utilizando pesos. Assim, sempre que uma fórmula envolver a contagem de exemplos, essa contagem deve ser substituída pela soma dos pesos dos exemplos correspondentes. Em outras palavras, a contribuição de cada exemplo para os cálculos deve ser ponderada pelo seu peso.

Para concluir este exercício, você escreverá código para implementar a floresta aleatória para esse problema:

1. Divisão de conjunto de dados:

- Carregue o conjunto de dados fornecido em seu código.
- Divida o conjunto de dados em três conjuntos: treinamento, validação e teste, com uma proporção de 70/15/15, respectivamente. Utilize `seed = 13`.

2. Implemente uma função para aprender Random Forest – as principais etapas são detalhadas abaixo:

- Inicialize uma floresta aleatória vazia.
- Determine o número de árvores de decisão que vão ser incluídas na floresta (por exemplo, 10) e o número de atributos aleatórios que vão ser considerados, geralmente $\text{num_features} \approx \sqrt{p}$ onde p é o número total de atributos.
- Implemente um loop para criar o número especificado de árvores de decisão:
 - Gere uma amostra de bootstrap a partir do conjunto de treinamento (amostragem com substituição).
 - Crie uma árvore de decisão usando o exemplo de bootstrap
 - Adicione a árvore de decisão construída à Floresta Aleatória.

3. Implemente uma função de classificação. Implemente uma função para classificar novas instâncias usando a Floresta Aleatória:

- Para cada instância, passe-a por todas as árvores de decisão na Floresta Aleatória e colete as previsões. Observe que você deve binarizar a previsão de cada árvore de decisão, ou seja, usar um limiar de 0.5 para determinar o rótulo de classe real.

- b. A previsão de cada árvore da floresta corresponderá à classe majoritária da folha com maior probabilidade de ser alcançada pelo exemplo. A previsão da floresta aleatória será calculada como a fração de árvores que preveem cada classe.

4. Execute seu algoritmo e avalie seu desempenho:

- a. Chame a função para aprender a Floresta Aleatória com seu conjunto de treinamento. Você variará os diferentes parâmetros da Floresta Aleatória (número de árvores e profundidade das árvores) para observar seu efeito no desempenho no conjunto de validação. Você usará o conjunto de treinamento para aprender a árvore e o conjunto de validação usando a Área sob a Curva Roc (AUROC) para encontrar os parâmetros ideais. Mantenha árvores de profundidade não superior a 5 e num_trees entre 5 e 20.
- b. Crie uma Floresta Aleatória usando os conjuntos de treinamento + validação com a melhor combinação de parâmetros.
- c. Classifique as instâncias do conjunto de testes usando a Floresta Aleatória, calcule a acurácia (limiar 0.5) e a Área sob a Curva ROC (AUROC) e relate os resultados.

5. Experimentação: Compare o desempenho de Florestas Aleatórias usando as duas estratégias diferentes para lidar com valores ausentes, e relate o desempenho no conjunto de testes em uma tabela.

IMPORTANTE: Não use implementações prontas de árvore de decisão (scikit-learn), Random Forest, GridSearchCV ou ferramentas equivalentes de busca automática.

Parte 2 – – Agrupamento com K-means (valor: 40%)

Neste exercício, você explorará o clustering implementando o algoritmo K-means. Você escreverá código para executar o agrupamento K-means enquanto visualiza o movimento dos centroides em cada iteração.

Para concluir este exercício, você escreverá código para implementar K-means para clustering:

1. **Preparação do conjunto de dados:** execute as células fornecidas no bloco de anotações que geram os pontos de dados artificiais para este exercício.
2. **K-means Clustering:**
 - a. Inicialize centroides de cluster K selecionando pontos K do conjunto de dados aleatoriamente.
 - b. Implemente um loop para executar as seguintes etapas até a convergência (ou até que um número máximo especificado de iterações seja atingido, por exemplo, 150):

- i. Atribua cada ponto de dados ao centroide mais próximo (você terá que calcular a distância euclidiana entre o ponto de dados e cada centroide).
- ii. Atualize cada centroide movendo-o para a média de todos os pontos de dados atribuídos a ele.
- iii. Verifique a convergência comparando os novos centroides com os centroides anteriores. Se a diferença for menor que um $\epsilon = 1e - 4$, saia do loop.

3. Visualização do movimento centroide:

- a. Em 5 momentos diferentes durante o treinamento, plote uma figura mostrando os centroides e os pontos. A Figura 1 deve mostrar a situação no início, antes do aprendizado. A Figura 5 deve mostrar a situação ao final do aprendizado. As Figuras 2 a 4 restantes devem mostrar situações intermediárias (veja um exemplo abaixo e observe que os pontos de dados usados aqui são diferentes dos que você usará, estes foram gerados para fins de ilustração).
- b. Para cada figura, cada centroide será representado por uma grande cruz preta e cada cluster com uma cor diferente, os pontos devem ser coloridos de acordo com seu respectivo agrupamento.

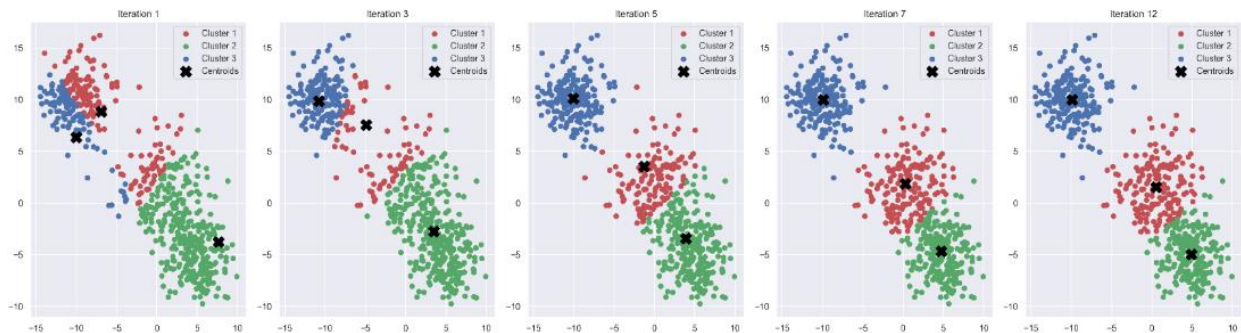
4. Soma das distâncias quadradas:

- a. Juntamente com a plotagem do movimento do centroide, calcule a soma das distâncias ao quadrado em cada iteração da seguinte maneira:

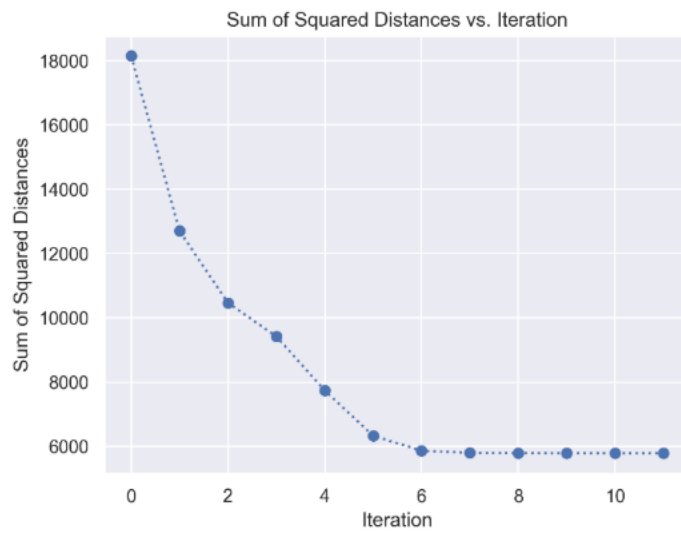
$$\sum_{j=1}^K \sum_{n \in S_j} d(x_n, \mu_j)^2$$

onde K é o número de clusters, x_n representa o ponto n^{th} de dados, $n \in S_j$ indica um conjunto de pontos que pertencem ao cluster S_j , μ_j é a média dos pontos de dados em S_j e $d(x_n, \mu_j)$ indica a distância euclidiana entre x_n e μ_j .

- b. Faça um gráfico da soma das distâncias ao quadrado em cada iteração (veja um exemplo abaixo, novamente, observe que os dados usados para esse gráfico são diferentes dos seus).



Exemplo da Visualização do Movimento Centróide



Exemplo da Soma de Distâncias Quadradas entre iterações